

Introdução à Programação



Conceitos Básicos de Orientação a Objetos

Construtor

- ◆ Tipo especial de método que é chamado no ato de criação de um objeto da classe desejada
 - Importante para fazer algumas inicializações no objeto
- ◆ Uma classe pode ter mais de um construtor
 - O que diferencia um construtor do outro é a ordem e/ou tipo e/ou quantidade de parâmetros
- ◆ Em C++, deve ter o mesmo nome da classe
- ◆ Não possui tipo de retorno
- ◆ Não é obrigatório definir um construtor
 - Existe em C++ um construtor default sem parâmetros

Implementando o Construtor do Tipo Conta

```
Conta::Conta(char *num, char* ag) {  
    strcpy(numero, num);  
    strcpy(agencia, ag);  
    saldo = 0;  
}
```

Usando o Tipo Conta em Programação OO

Comando para criar objeto de uma classe em tempo de execução (dinamicamente)

Para imprimir na tela

```
#include <iostream>
using namespace std;
int main() {
    Conta *c;
    c = new Conta("1234-5", "007-x");
    if (c == NULL) {
        printf("Erro na criação do objeto!")
    } else {
        c->creditar(300);
        cout<<"Conta tem saldo = " << c->getSaldo() << endl;
        c->debitar(200);
        cout<<"Conta tem saldo = " << c->getSaldo() << endl;
    }
    return 0;
}
```

Para chamar método a partir de uma referência para um objeto

Criação Dinâmica de Objetos

- ◆ Em linguagens OO, escrevemos programas utilizando objetos
- ◆ Mas, primeiro, temos que **criá-los**...
- ◆ Para criar um objeto em C++ dinamicamente utilizamos o comando ***new***

```
new Conta ( . . . ) ;
```

Palavra
reservada
ordenando a
criação
de um objeto

Construtor do
objeto a ser
criado

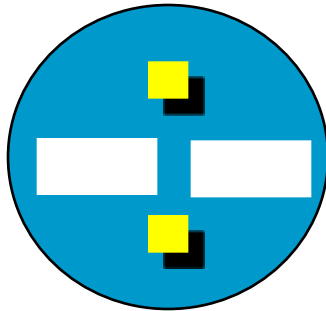
Avaliação do operador new

Para avaliar uma expressão do tipo
new NomeDaClasse () ;
o computador...

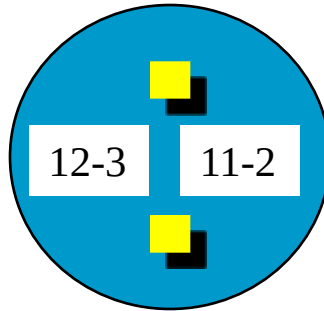
- ◆ Cria um objeto da classe **NomeDaClasse** e armazena na sua memória
- ◆ Inicializa os atributos deste objeto usando o construtor desta classe
- ◆ Devolve como resultado da avaliação uma referência (endereço da memória) para o objeto criado

Criando um objeto dinamicamente...

```
new Conta("12-3", 11-2) ;
```

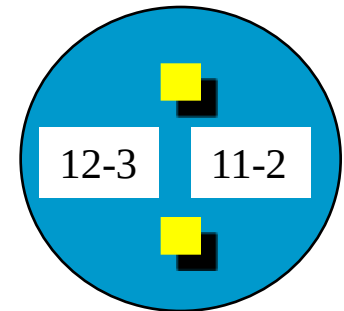


1. Cria o objeto
na memória



2. Inicializa o
objeto

→
id275



3. Associa um
endereço ao
objeto

4. Devolve o endereço
como resultado da avaliação

Construtores e new

- ◆ Podemos ter **mais** de um construtor para um determinado objeto
- ◆ A escolha do construtor para inicializar os atributos é determinada pela lista de **argumentos**, entre parênteses

```
new NomeDaClasse(argumentos)
```

O número, ordem e tipos
dos argumentos
determina o construtor

Referências para objetos

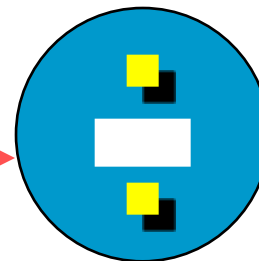
- ◆ Todo acesso e manipulação é feito indiretamente, através de uma **referência (endereço)** para o objeto
- ◆ Quando declaramos uma variável do tipo ponteiro para um objeto, ela não armazena o objeto e sim uma referência para um objeto daquele tipo

```
Conta* conta1 = new Conta("12-3", "11-2");
```

A referência
para o objeto

conta1 armazena
este valor

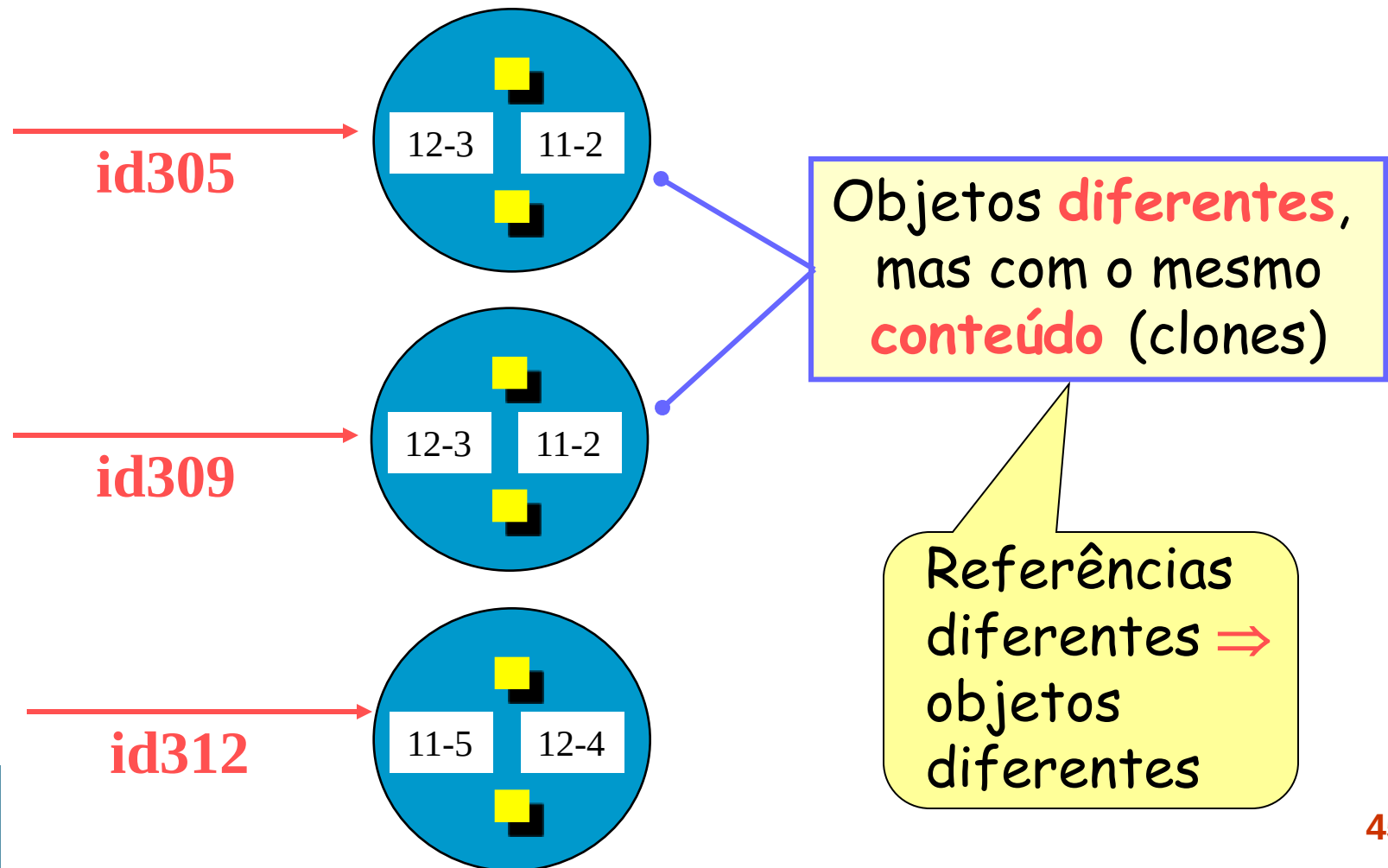
id305



O objeto

A referência funciona
como a **identidade** do objeto

Referências e identidade de um objeto



Considerações sobre Criação de Objetos

- ◆ Os valores dos atributos (estado) de um objeto podem mudar durante a execução de um programa, mas isso não afeta a identidade do objeto
- ◆ O ato de criar um objeto é chamado de **instanciar** um objeto de uma determinada classe

Criação Estática de Objetos

- ◆ C++ permite também a criação estática (em tempo de compilação) de objetos
- ◆ No ato da declaração de uma variável do tipo de uma classe, objeto é criado
 - Na declaração deve-se passar argumentos do construtor

```
Conta c ("12-3", "11-2") ;
```

Tipo da
variável **c** é
Conta

Argumentos do
construtor

Usando Criação Estática de Objetos do Tipo Conta

Declaração para instanciar objeto em tempo de compilação

```
#include <iostream>
using namespace std;
int main() {
    Conta c("1234-5", "007-x");
    c.creditar(300);
    cout<<"Conta tem saldo = " << c.getSaldo() << endl;
    c.debitar(200);
    cout<<"Conta tem saldo = " << c.getSaldo() << endl;

    return 0;
}
```

Utiliza-se operador "." para utilizar os métodos do objeto criado estaticamente